Продвинутое использование git

Лабораторная работа № 4 (зачётная книжка № 1132259051)

Чжан Вэньцзюнь

2026-10-20

Содержание

1 Цель работы **5**

2 Задание **6**

3 Теоретическое введение **7**

4 Выполнение лабораторной работы **8**
4.1 Установка программного обеспечения 8
4.2 Настройка Node.js 9
4.3 Общепринятые коммиты 9
4.4 Создание тестового репозитория 10
4.5 Практический сценарий использования git 11

5 Выводы **13**

Список литературы **14**

Список иллюстраций

4.1 Установленные инструменты 9
4.2 Файл package.json, теги и ветки 10
4.3 История репозитория после выпусков 12

Список таблиц

1 Цель работы

Получение навыков правильной работы с репозиториями git: освоение рабочего процесса Gitflow, семантического версионирования и общепринятых коммитов.

2 Задание

1. Выполнить работу для тестового репозитория.
2. Преобразовать рабочий репозиторий в репозиторий с git-flow и conventional commits.

3 Теоретическое введение

Рабочий процесс Gitflow, опубликованный Винсентом Дриссеном, предполагает строгую модель ветвления с учётом выпуска проекта: в ветке `master` хранится официальная история релизов, ветка `develop` предназначена для объединения всех функций, под каждую функцию отводится отдельная ветка `feature`, для подготовки выпуска — ветка `release`, для срочных исправлений — `hotfix`.

Семантическое версионирование задаёт формат номера версии `MAJOR.MINOR.PATCH`, в котором мажорный номер увеличивается при несовместимых изменениях API, минорный — при добавлении функциональности с обратной совместимостью, патч — при исправлении ошибок.

Общепринятые коммиты (Conventional Commits) задают формат сообщения [illegible], где тип принимает значения `feat`, `fix`, `docs`, `chore` и другие. Такой формат позволяет автоматически формировать журнал изменений [1].

4 Выполнение лабораторной работы

4.1 Установка программного обеспечения

Установлен пакет `gitflow` из репозитория Copr и среда Node.js, необходимая для работы утилит семантического версионирования и общепринятых коммитов:

```
sudo dnf -y copr enable elegos/gitflow
sudo dnf -y install gitflow
sudo dnf -y install nodejs24 nodejs24-npm pnpm
```

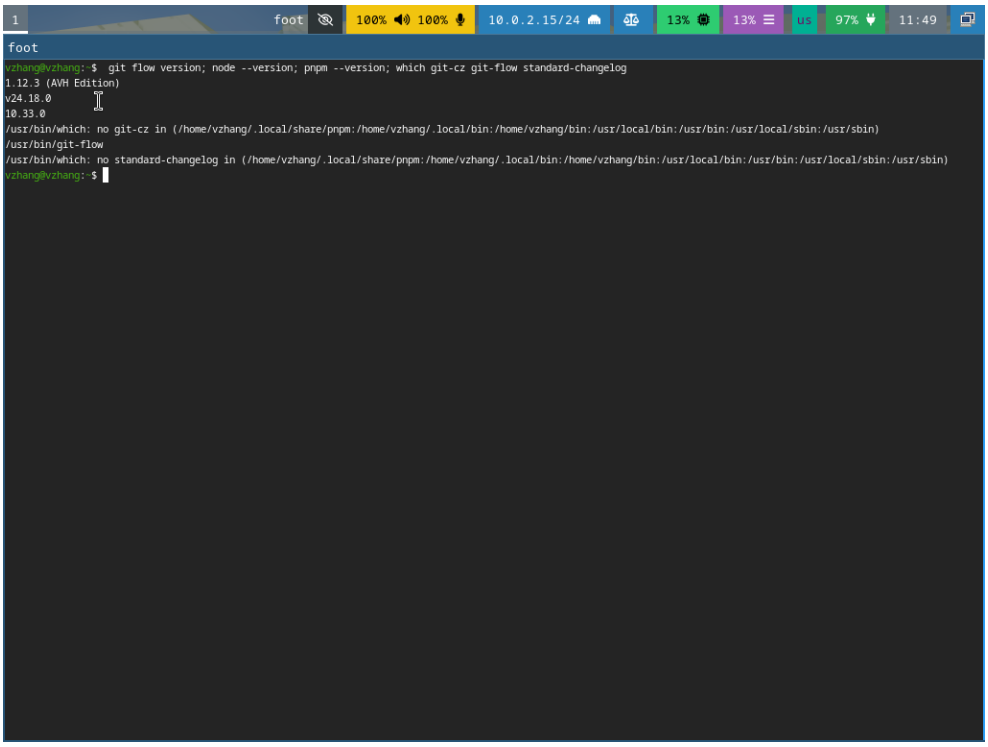


Рисунок 4.1: Установленные инструменты

4.2 Настройка Node.js

Каталог с исполняемыми файлами пакетов, устанавливаемых через `pnpm`, добавлен в переменную PATH:

```
pnpm setup
source ~/.bashrc
```

4.3 Общепринятые коммиты

Установлены утилиты `commitizen` (скрипт `git-cz` для форматирования сообщений коммитов) и `standard-changelog` (формирование журнала изменений):

```
npm install -g commitizen standard-changelog
```

При установке из-за низкой скорости официального реестра использовано зеркало `registry.npmmirror.com`.

4.4 Создание тестового репозитория

Создан каталог `~/work/study/2026-2027/git-extended`, инициализирован репозиторий, подготовлен файл `package.json` с настройкой `commitizen` и метаданными пакета:

```
mkdir git-extended && cd git-extended
git init -b master
```

Содержимое `package.json` приведено на Рисунок 4.2; там же показаны созданные теги и ветки.

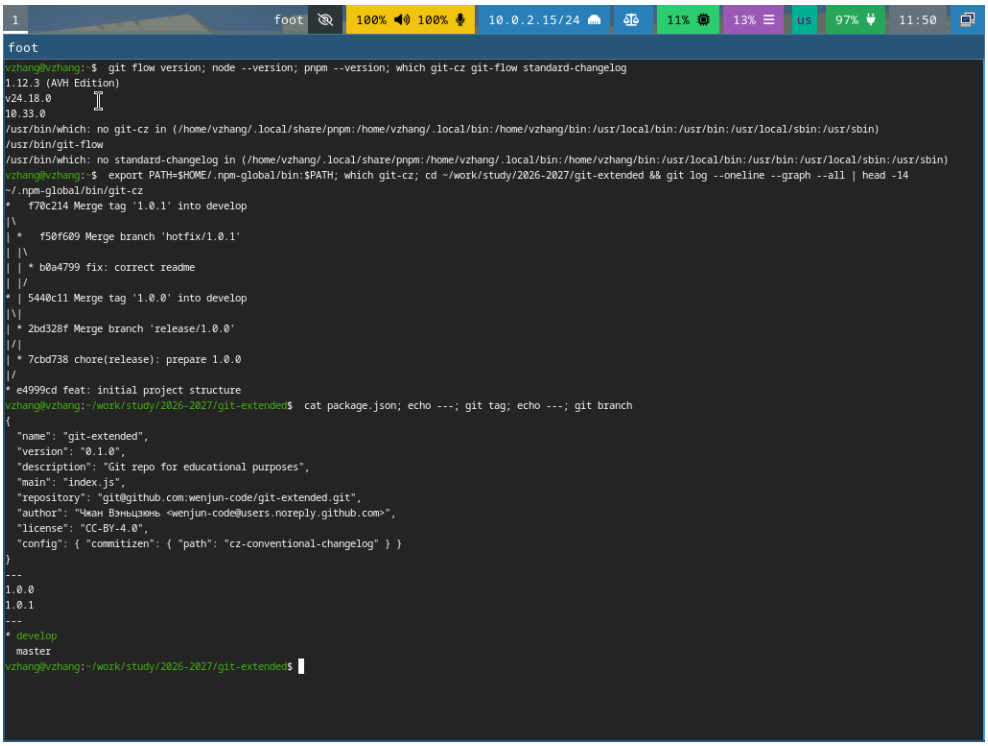


Рисунок 4.2: Файл package.json, теги и ветки

4.5 Практический сценарий использования git

Инициализирована структура Gitflow с параметрами по умолчанию (`-d`), в результате созданы ветки `master` и `develop`. Затем выполнены выпуск версии 1.0.0 и исправление 1.0.1:

```
git flow init -d
git flow release start 1.0.0
git add VERSION && git commit -m 'chore(release): prepare 1.0.0'
git flow release finish -m 'chore(release): 1.0.0' 1.0.0
git flow hotfix start 1.0.1
git commit -am 'fix: correct readme'
git flow hotfix finish -m 'fix: 1.0.1' 1.0.1
```

Сообщения коммитов оформлены в соответствии с Conventional Commits: `feat: initial project structure`, `chore(release): prepare 1.0.0`, `fix: correct readme`.

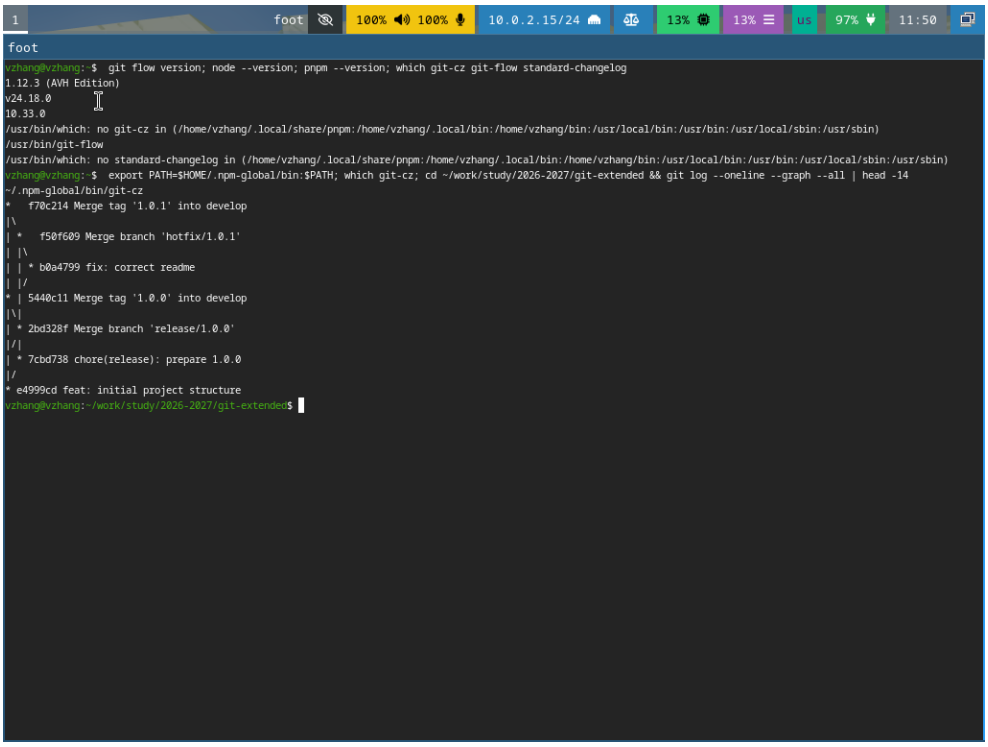


Рисунок 4.3: История репозитория после выпусков

В результате в репозитории созданы два тега (`1.0.0` и `1.0.1`), ветки `master` и `develop`, а история содержит слияния веток `release/1.0.0` и `hotfix/1.0.1`.

5 Выводы

В ходе работы освоен рабочий процесс Gitflow: инициализация структуры веток, подготовка и завершение выпуска, выполнение срочного исправления. Изучено семантическое версионирование и формат общепринятых коммитов. Установлены и применены утилиты `gitflow`, `commitizen` и `standard-changelog`. Тестовый репозиторий преобразован в репозиторий с git-flow и conventional commits. Все задачи лабораторной работы выполнены.

Список литературы

1. Robbins A. Bash Pocket Reference. O'Reilly Media, 2016. 156 c.